

CAGOULE

v1.2.0

Rapport Technique de Projet

Cryptographie Algébrique Géométrique par Ondes et Logique Entrelacée

Système de chiffrement symétrique hybride

fondé sur les mathématiques du Concours Général Sénégalais 2025

Auteur	Slim Issa (slimissa)
Version	1.2.0 — publiée sur PyPI
Licence	MIT
Langages	Python 99.1% · C 0.2%
GitHub	github.com/slimissa/CAGOULE
Date	Avril 2026

1. Présentation du projet

CAGOULE (Cryptographie Algébrique Géométrique par Ondes et Logique Entrelacée) est un système de chiffrement symétrique hybride conçu par Slim Issa. Le projet fusionne des primitives cryptographiques modernes standardisées (ChaCha20-Poly1305, Argon2id, HKDF) avec des structures mathématiques issues du Concours Général Sénégalais 2025 (CGS2025).

3 187
lignes de code

13
modules Python

11
classes définies

v1.2.0
publiée sur PyPI

2. Architecture du pipeline de chiffrement

Le chiffrement s'effectue en deux couches successives :

Couche 1 — CBC-like interne (algèbre mod p)

- Le plaintext est padé PKCS7 et découpé en blocs de 16 octets
- Chaque octet devient un élément de $\mathbb{Z}/p\mathbb{Z}$ ($p \approx 2^{64}$, premier dérivé depuis le mot de passe)
- Pour chaque bloc m_i : $v = m_i + \text{prev_cipher} \bmod p$ (CBC mixing)
- $w = P \times v \bmod p$ — P : matrice de Vandermonde 16×16 (diffusion)
- $u = \text{S-box}(w)$ — $f(x) = x^3 + cx$, bijective sur $\mathbb{Z}/p\mathbb{Z}$ (confusion)
- $c = u + \text{round_key} \bmod p$ — round key dérivée de $\zeta(2n)$ (Fourier $\rightarrow$ HKDF)

Couche 2 — Enveloppe AEAD (ChaCha20-Poly1305)

- La sortie $T(\text{message})$ est chiffrée par ChaCha20-Poly1305 (RFC 8439)
- K_{stream} (256 bits) dérivée de K_{master} via HKDF
- $\text{AAD} = \text{Magic} \parallel \text{Version} \parallel \text{Salt}$ — authentifié mais transmis en clair
- Tag Poly1305 de 16 octets — garantie d'intégrité et d'authenticité

3. Analyse des modules

Le projet compte 3 187 lignes réparties en 13 modules, organisés en 4 couches fonctionnelles.

Fichier	Couche	Rôle	Total	Code	Classes	Fonctions
cipher.py	Core	Chiffrement CBC-like + AEAD	247	173	0	2
decipher.py	Core	Déchiffrement inverse + exceptions	237	174	3	2
params.py	Core	Dérivation paramètres (KDF $\rightarrow$ tout)	305	228	1	1
omega.py	Math	Pilier Ω : $\zeta(2n) \rightarrow$ round keys	228	155	1	1
sbox.py	Math	S-box x^3+cx / fallback x^d	308	237	1	1

Fichier	Couche	Rôle	Total	Code	Classes	Fonctions
matrix.py	Math	Matrices Vandermonde / Cauchy mod p	307	248	2	2
mu.py	Math	Racine de x^4+x^2+1 ($\mathbb{Z}/p\mathbb{Z}$ ou $\mathbb{F}_{p^2}$)	298	225	1	0
fp2.py	Math	Extension quadratique $\mathbb{F}_{p^2}$	283	236	1	0
constants.py	Math	Constantes CGS2025 (ζ , ρ , β , x_0)	152	94	0	0
format.py	Format	Parsing / sérialisation CGL1	346	265	2	6
cli.py	CLI	Interface ligne de commande	399	309	0	5
__init__.py	Package	API publique	71	57	0	0
__version__.py	Package	Métadonnées version	6	6	0	0
TOTAL		13 modules	3 187	2 606	11	20+

4. Primitives cryptographiques

Primitive	Algorithme	Paramètres
KDF principal	Argon2id	$t=3$, $m=64$ MB, $p=1$
KDF fallback	Scrypt	$n=2^{17}$, $r=8$, $p=1$
AEAD	ChaCha20-Poly1305	RFC 8439 — nonce 96 bits
Dérivation clés	HKDF-SHA256	RFC 5869
Chiffrement interne	CBC-like $\mathbb{Z}/p\mathbb{Z}$	Bloc 16 éléments, mod $p \approx 2^{64}$
Diffusion	Vandermonde / Cauchy	Matrice 16×16 mod p
Confusion	x^3+cx / x^d	Bijective sur $\mathbb{Z}/p\mathbb{Z}$, fallback x^d
Round keys	$\zeta(2n) \rightarrow \text{Fourier}$	64 clés, $n \in [4, 65536]$

5. Format binaire CGL1

Chaque message chiffré est sérialisé dans un format binaire propriétaire CGL1. L'overhead fixe est de 65 octets quel que soit la taille du plaintext.

Offset	Taille	Champ	Description
0 – 3	4 octets	Magic	CGL1 (0x43474C31) — identifiant du format
4	1 octet	Version	0x01 — version du protocole
5 – 36	32 octets	Salt	Sel Argon2id (inclus dans AAD)

Offset	Taille	Champ	Description
37 – 48	12 octets	Nonce	Nonce ChaCha20 (96 bits, aléatoire)
49 – N	variable	CT	Ciphertext T(message) chiffré par ChaCha20
N – N+16	16 octets	Tag	Tag Poly1305 (128 bits) — authentication AEAD
Overhead fixe	65 octets = 4+1+32+12+16		

6. Constantes mathématiques CGS2025

CAGOULE intègre les résultats mathématiques du Concours Général Sénégalais 2025. Toutes les constantes sont calculées avec mpmath à 60 chiffres significatifs.

Constante	Formule	Valeur numérique
ρ (nombre d'or)	$(1 + \sqrt{5}) / 2$	$\approx 1.61803398874989484820\dots$
β (volume solide S)	$(8\pi/81)(55\rho + 34)$	$\approx 34.21\dots$
$\Omega = \zeta(8)$	$\pi^8 / 9450$	$\approx 1.00407735619794433937\dots$
x_0	3π	$\approx 9.42477796076937971538\dots$
$\delta = (Z/13Z)^* $	$p - 1$ pour $p = 13$	$= 12$ (ordre du groupe multiplicatif)

Identité clé — vérifiée à 60 décimales : $\zeta(8) = \pi^8/9450 \approx 1.0040773561979443393\dots$

Cette identité est exploitée pour générer les 64 round keys via les coefficients de Fourier $a_k = (2/\pi) \times (-1)^{(k+1)/2} / k^{(2n)}$, avec $n \in [4, 65536]$ dérivé du mot de passe.

7. Pipeline de dérivation des paramètres

Tous les paramètres sont dérivés déterministement depuis le couple (password, salt) :

- password + salt $\rightarrow$ Argon2id ($t=3$, $m=64$ MB) $\rightarrow$ K_master (512 bits)
- K_master + 'CAGOULE_N' $\rightarrow$ HKDF $\rightarrow$ $n \in [4, 65536]$ (paramètre ζ)
- K_master + 'CAGOULE_P' $\rightarrow$ HKDF $\rightarrow$ p_seed $\rightarrow$ nextprime(p_seed | 2^{63})
- p $\rightarrow$ generate_mu(p) $\rightarrow$ μ racine de x^4+x^2+1 (Stratégie A : $\mathbb{Z}/p\mathbb{Z}$, ou C : $\mathbb{F}_p^2$)
- K_master + 'CAGOULE_DELTA' $\rightarrow$ HKDF $\rightarrow$ $\delta \rightarrow$ SBox.from_delta(δ , p)
- μ + K_master $\rightarrow$ 16 nœuds distincts $\rightarrow$ DiffusionMatrix (Vandermonde 16×16)
- K_master + 'CAGOULE_ENC' $\rightarrow$ HKDF $\rightarrow$ K_stream (256 bits ChaCha20)
- n + salt + p $\rightarrow$ $\zeta(2n) \rightarrow$ 64 coefficients Fourier $\rightarrow$ HKDF $\rightarrow$ 64 round keys mod p

Entropie supplémentaire : n est secret (16 bits d'entropie). Sans n, les round keys sont mathématiquement irrécupérables.

8. Known Answer Test (KAT)

Le fichier `kat_vectors.json` contient le vecteur de test de référence v1.1, utilisé pour valider la non-régression entre implémentations.

Paramètre	Valeur
Version KAT	1.1
Mot de passe	CAGOULE_KAT_2026_MASTER_FIXED
Plaintext	"Hello, World!"
n_zeta dérivé	63 237
p dérivé	13 226 797 537 736 071 951 ($\approx 2^{63}$)
Stratégie μ	A — racine dans $\mathbb{Z}/p\mathbb{Z}$
Taille CGL1	193 octets
SHA-256 CGL1	87f3e70d...6b5c3f8

9. Carte des dépendances internes

Les modules sans dépendances internes (`fp2`, `sbox`, `matrix`, `constants`, `format`) constituent la base stable de la bibliothèque.

Module	Dépend de...
<code>cipher.py</code>	<code>params</code> · <code>omega</code>
<code>decipher.py</code>	<code>params</code> · <code>cipher</code> (<code>MAGIC</code> , <code>NONCE_SIZE...</code>) · <code>omega</code>
<code>params.py</code>	<code>fp2</code> · <code>mu</code> · <code>sbox</code> · <code>matrix</code> · <code>omega</code>
<code>omega.py</code>	— (<code>mpmath</code> , <code>cryptography</code> uniquement)
<code>sbox.py</code>	— (<code>stdlib</code> uniquement)
<code>matrix.py</code>	— (<code>stdlib</code> uniquement)
<code>mu.py</code>	<code>fp2</code>
<code>fp2.py</code>	— (<code>stdlib</code> uniquement)
<code>constants.py</code>	— (<code>mpmath</code> uniquement)
<code>format.py</code>	— (<code>stdlib</code> uniquement)
<code>cli.py</code>	<code>cipher</code> · <code>decipher</code> · <code>format</code> · <code>params</code>

Dépendances externes (PyPI)

Paquet	Version min	Usage
<code>cryptography</code>	≥ 42.0	ChaCha20-Poly1305, HKDF, Scrypt

Paquet	Version min	Usage
mpmath	≥ 1.3	Précision arbitraire pour $\zeta(2n)$ et constantes CGS2025
argon2-cffi	≥ 23.0	KDF principal Argon2id (fallback Scrypt si absent)
pytest	≥ 7.0	Suite de tests (dépendance dev)

10. État du dépôt GitHub

URL	github.com/slimissa/CAGOULE
Branche	master
Commits	4 commits
Contributeurs	1 (slimissa)
Tests badge	162 / 162 ✓
CI	GitHub Actions (.github/workflows/tests.yml)
PyPI	pypi.org/project/cagoule — v1.2.0 publiée
Licence	MIT
Langages	Python 99.1% · C 0.2% · CSS 0.5% · JS 0.1% · TeX 0.0%

11. Points d'amélioration identifiés

Priorité	Problème identifié	Action recommandée
☐ Haute	Structure plate à la racine (tous les .py au root, pas dans cagoule/)	Déplacer dans cagoule/ + imports relatifs
☐ Haute	dist/ et venv/ committés dans le dépôt	Ajouter au .gitignore, nettoyer l'historique
☐ Moyen	sbox_inverse_exhaustive : $O(p)$ pour $p \approx 2^{64}$	Vérifier que Newton est systématiquement utilisé
☐ Moyen	Pas de tests unitaires visibles pour fp2.py et mu.py	Ajouter test_fp2.py et test_mu.py
☐ Basse	pyproject.toml : build-backend non standard	Utiliser setuptools.build_meta directement
☐ Basse	cagoule.egg-info/ committé	Ajouter au .gitignore

12. Conclusion

CAGOULE v1.2.0 est un projet cryptographique expérimental ambitieux et bien structuré. Les points forts sont :

- Double couche de sécurité : algèbre custom + ChaCha20-Poly1305 (standard industriel)
- Originalité mathématique : intégration du CGS2025 (ζ , Fourier, Vandermonde, Fp^2)
- Robustesse par design : fallbacks automatiques (SBox x^d , Cauchy, Fp^2 pour μ)
- Pipeline de dérivation solide : Argon2id $\rightarrow$ K_master $\rightarrow$ tous les paramètres
- Publié sur PyPI avec CI GitHub Actions et 162 tests

Les priorités immédiates sont la restructuration du dépôt (imports relatifs, nettoyage du .gitignore) et l'extension de la suite de tests pour les modules fp2.py et mu.py.

*CAGOULE est à but académique et expérimental. **Ne pas utiliser en production sans audit de sécurité indépendant.***